PHASE 1A: EVENT TICKETING PLATFORM

Sections 10, 11 & 12 — Testing, Deployment & Fundamentals

Testing Strategy • Deployment Guide • System Design Study Approach

This document covers three operational tracks for Phase 1A: how to test the system with confidence, how to deploy it to production, and how to use the project to build system design thinking. Code blocks have been converted to structured prose and reference tables. All class names, annotation names, CLI commands, and configuration keys are preserved inline throughout.

SECTION 10

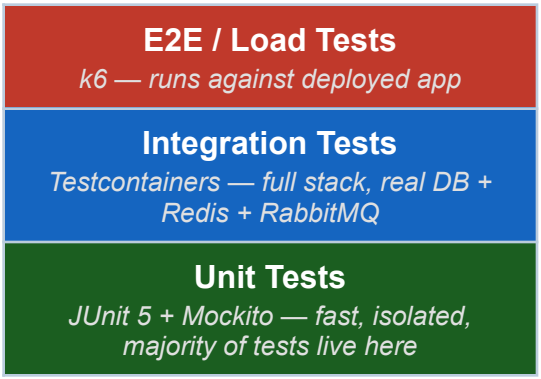
Testing Strategy

Test pyramid, unit test patterns, Testcontainers integration setup, coverage targets, and JaCoCo configuration

Testing is not an afterthought in this project — it is woven into each day's work. The goal is **confidence, not coverage for its own sake**. Every test must answer a real question: "Does this behaviour work?" The pyramid below shows where effort is concentrated.

10.1 The Test Pyramid

Tests are organized into three layers. The pyramid shape reflects where **most tests should live** — at the bottom (fast, isolated unit tests), not at the top (slow, brittle E2E tests).



Layer	Tool(s)	When to Write / What it Tests
Unit Tests	JUnit 5 + Mockito	Write first (TDD). Tests a single class in complete isolation — all dependencies mocked. Runs in < 1 second each. Lives alongside production code. 80%+ of all tests should be here.
Integration Tests	Testcontainers + @SpringBootTest	Write after the feature is unit-tested. Tests multiple layers together with real infrastructure (real Postgres, real Redis, real RabbitMQ in Docker containers). Slow to start but deterministic.
E2E / Load Tests	k6	Write in Week 3, run against the locally deployed app. Tests the system under concurrent load — 1,000 virtual users, verifies no double-bookings, measures P95 response time.

10.2 Unit Test Patterns

All unit tests use the **Arrange / Act / Assert** (AAA) structure. Every test class is annotated `@ExtendWith(MockitoExtension.class)`. The class under test is annotated `@InjectMocks`. Every dependency is annotated `@Mock`.

PricingEngineTest — 100% Branch Coverage Required

The `PricingEngine` has no dependencies — it is a pure function class. Instantiate it directly with `private PricingEngine pricingEngine = new PricingEngine()`. No mocking needed.

Test Method	Scenario and Expected Result
`earlyBird_30DaysOut_shouldApply50PercentDiscount`	Call calculatePrice() with a $100 base price, quantity 1, event date 30 days from now, 0 sold. Assert result equals "50.00" (50% off).
`groupDiscount_5OrMoreTickets_shouldApply10PercentOff`	Call calculatePrice() with $100 base price, quantity 5, event 60 days away, 0 sold. Assert result equals "450.00" (5 × $100 × 0.90).
`dynamicPricing_over80PercentSold_shouldApply25PercentSurge`	Set capacity=100 and currentSoldCount=85 (85%). Call calculatePrice() with $100 base, quantity 1. Assert result equals "125.00" (25% surge).
`combinedDiscounts_earlyBirdAndGroup_bothApply`	Event 30+ days away, quantity 5. Both early-bird (50%) and group (10%) discounts should stack. Assert correct compounded total.
`noDiscount_regularPurchase_returnsBasePrice`	Event < 30 days away, quantity 1, < 80% sold. Assert result equals base price with no modification.
`refund_7OrMoreDaysBeforeEvent_shouldBeFullAmount`	Booking with totalAmount=$200, event starts in 10 days. Call calculateRefundAmount(). Assert result equals "200.00".
`refund_3To7DaysBeforeEvent_shouldBe50Percent`	Booking with totalAmount=$200, event starts in 5 days. Assert result equals "100.00" (50%).
`refund_lessThan3DaysBeforeEvent_shouldBeZero`	Booking with totalAmount=$200, event starts in 2 days. Assert result equals "0.00" — no refund.

> **i NOTE:** Why 100% branch coverage on PricingEngine? It contains 4 independent pricing rules and 3 refund tiers — each is an if/else branch. A missed branch means a pricing rule was never exercised by a test. 100% branch coverage guarantees every rule path is correct.

General Unit Test Rules

- Use `@DisplayName` on every test with a human-readable sentence: `"AVAILABLE → RESERVED: should acquire Redis lock and decrement inventory"`. Test reports should be readable without decoding method names.
- Use AssertJ (`assertThat()`) over JUnit `assertEquals()` — the fluent API produces far more descriptive failure messages.
- For `BigDecimal` assertions, always use `.isEqualByComparingTo("50.00")` instead of `.isEqualTo()` — the latter fails on scale differences (`50.00` ≠ `50.0` with equals, but both pass isEqualByComparingTo).

- Never put business logic in a test helper method. Each test's `// Arrange` block should be explicit — a reader should understand exactly what state is being set up without navigating to helper methods.
- Name test helper factory methods descriptively: `bookingWithAmount(BigDecimal amount)`, `tier(BigDecimal basePrice)`, `confirmedBooking(Long bookingId)` — these replace builder boilerplate and keep test bodies concise.

10.3 Testcontainers Integration Test Setup

All integration tests extend a single `BaseIntegrationTest` abstract class. This class starts one set of Docker containers that are **shared across all integration test methods** in the test run — not restarted per test. This is the critical optimisation that keeps the integration test suite from being unbearably slow.

BaseIntegrationTest.java — Class-Level Setup

- **Class annotations:** `@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)` — starts the full Spring context with a real HTTP server on a random port; `@Testcontainers` — activates Testcontainers lifecycle management; `@ActiveProfiles("test")` — loads `application-test.yml` which expects Testcontainers-injected connection properties.
- **Container fields are `static` — this is essential**. Static containers are started once per JVM process and shared by every test class that extends `BaseIntegrationTest`. Non-static containers are started and stopped per test class, which is orders of magnitude slower.

Container Declarations

- **PostgreSQL:** `static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15-alpine")` with `.withDatabaseName("ticketing_test")`, `.withUsername("test")`, `.withPassword("test")`.
- **Redis:** `static GenericContainer<?> redis = new GenericContainer<>("redis:7-alpine")` with `.withExposedPorts(6379)` and `.waitingFor(Wait.forLogMessage(".*Ready to accept connections.*\n", 1))`. The `waitingFor` call ensures the container is fully started before Spring attempts connection — without this, the app may connect before Redis is ready.
- **RabbitMQ:** `static RabbitMQContainer rabbitmq = new RabbitMQContainer("rabbitmq:3.12-management-alpine")`. The Testcontainers `RabbitMQContainer` provides helper methods `getHost()` and `getMappedPort(5672)`.

@DynamicPropertySource — Wiring Containers to Spring

The `@DynamicPropertySource` static method injects the container's host and mapped port into Spring's `Environment` before the application context is created. This overrides `application-test.yml` values at test runtime. The registry adds these properties:

- `spring.datasource.url` → `postgres.getJdbcUrl()`

- `spring.datasource.username` → `postgres.getUsername()`
- `spring.datasource.password` → `postgres.getPassword()`
- `spring.data.redis.host` → `redis.getHost()`
- `spring.data.redis.port` → `redis.getMappedPort(6379).toString()`
- `spring.rabbitmq.host` → `rabbitmq.getHost()`
- `spring.rabbitmq.port` → `rabbitmq.getMappedPort(5672).toString()`

Using BaseIntegrationTest

- Extend `BaseIntegrationTest` for any test that needs the real DB, Redis, or RabbitMQ. Inject Spring beans with `@Autowired`.
- For test isolation: use `@Transactional` on integration test methods to automatically roll back every database change after each test — no manual cleanup needed.
- For RabbitMQ consumer tests: use Awaitility to wait for asynchronous message processing — `Awaitility.await().atMost(5, TimeUnit.SECONDS).until(() -> someCondition)`. Never use `Thread.sleep()`.

10.4 Coverage Targets and JaCoCo Configuration

Coverage targets are enforced by JaCoCo as part of the Maven build. A failing coverage check causes `./mvnw verify` to fail — the same command run by GitHub Actions CI. This prevents coverage from silently regressing.

Coverage Targets by Module

Module / Class	Instruction %	Branch %	Rationale
Overall project (BUNDLE)	**≥ 80%**	**≥ 75%**	Enforced by JaCoCo check goal — build fails if not met
PricingEngine.java	**100%**	**100%**	Every pricing rule and refund tier must be exercised
BookingStateMachine (config + actions)	**100%**	**100%**	Every state transition path must be tested
RefundService.java	**100%**	**100%**	Every refund tier (full, partial, zero) and error path must be covered
BookingService.java	**≥ 90%**	**≥ 85%**	Core business logic — high confidence required
PaymentService.java	**≥ 85%**	**≥ 80%**	Stripe integration — all success and failure paths
Controllers	**≥ 70%**	**≥ 65%**	Covered by @WebMvcTest layer — auth + happy + error paths

Module / Class	Instruction %	Branch %	Rationale
Config classes (RedisConfig, RabbitMQConfig)	**Excluded**	**Excluded**	Infrastructure wiring — not meaningful to unit test

Generating and Reading the JaCoCo Report

- **Generate:** Run `./mvnw clean test jacoco:report`. After the build completes, open `target/site/jacoco/index.html` in a browser.
- **Reading the report:** Green lines = covered. Red lines = never executed. Yellow lines = partially covered (some branches missed). The **Branches** column is the most important — it catches every if/else decision point.
- **Finding gaps efficiently:** Click into the `PricingEngine` class first. Red lines show which pricing rule or refund condition was never triggered. Write a test that enters that branch.
- **Excluding classes from coverage:** Add `@Generated` annotation on classes you want JaCoCo to skip (config classes, generated code). Configure the `<excludes>` element in the JaCoCo `<configuration>` block in `pom.xml`.

JaCoCo Maven Plugin — pom.xml Configuration

Add the `jacoco-maven-plugin` version `0.8.11` to the `<build><plugins>` section of `pom.xml`. Three executions are required:

- **prepare-agent execution:** Goal `prepare-agent`. No phase or configuration needed. Instruments the bytecode so coverage data is collected during `mvn test`.
- **report execution:** Goal `report`, phase `test`. Generates the HTML report in `target/site/jacoco/` after tests run.
- **check execution:** Goal `check`. Configures the enforcement rule: element `BUNDLE`, counter `INSTRUCTION`, value `COVEREDRATIO`, minimum `0.80`. If coverage falls below 80%, `./mvnw verify` fails.

10.5 Complete Test Class Inventory

Every test class that must exist by Day 21 for Phase 1A completion:

Test Class	Type	What It Covers
`EventServiceTest`	Unit	All 5 CRUD operations, organizer authorization, DRAFT→PUBLISHED transition — 8+ tests
`EventControllerTest`	Unit (@WebMvcTest)	All 5 endpoints: happy path, validation errors, auth failures (no token, wrong role)

Test Class	Type	What It Covers
`EventIntegrationTest`	Integration (Testcontainers)	Full CRUD + search with real PostgreSQL — extends BaseIntegrationTest
`BookingStateMachineTest`	Unit	5 state transitions, guard blocking, invalid transition exception — 5+ tests
`ConcurrentBookingTest`	Integration (Testcontainers)	100 threads simultaneous reserve → exactly 50 succeed, 50 fail, zero oversell
`PaymentServiceTest`	Unit	createCheckoutSession(), handlePaymentSuccess(), handlePaymentExpired() — 4+ tests with Stripe SDK mocked
`PaymentWebhookTest`	Unit	Valid signature processed, invalid signature returns 400, duplicate event returns 200 (idempotency)
`PricingEngineTest`	Unit	All 8 pricing and refund scenarios — 100% branch coverage required
`InventoryServiceTest`	Integration (Testcontainers Redis)	getAvailableCount(), reserveSeat(), releaseSeat(), concurrency test — 4+ tests
`NotificationListenerIntegrationTest`	Integration (Testcontainers RabbitMQ)	Publish BookingConfirmedEvent → Awaitility wait → verify email sent (mocked EmailService)
`WaitlistIntegrationTest`	Integration (Testcontainers Redis)	joinWaitlist(), notifyNextInWaitlist(), verify WaitlistAvailableEvent published
`RefundServiceTest`	Unit	3 refund tiers (full/partial/zero), Stripe Refund.create() called correctly — 5+ tests
`BookingControllerTest`	Unit (@WebMvcTest)	All booking endpoints: happy path, auth failures, ownership check

SECTION 11

Deployment Guide

Docker Compose local environment, Railway production deployment, environment variables, and health verification

Deployment follows a two-environment model: **local development** via Docker Compose (single command, all services, no cloud account needed), and **production** via Railway (automatic deployment from GitHub, managed database services). The same Docker image runs in both environments.

11.1 Service Port Reference

All ports used by the local Docker Compose environment. Memorise these — you will use them daily during development.

Service	Port	Protocol	Access URL / Purpose
Spring Boot (app)	`8080`	HTTP	http://localhost:8080 — API endpoints + /actuator/health
Next.js (frontend)	`3000`	HTTP	http://localhost:3000 — the web UI
PostgreSQL	`5432`	TCP	localhost:5432 — connect via pgAdmin or any DB client
Redis	`6379`	TCP	localhost:6379 — use redis-cli or redis-commander
RabbitMQ (AMQP)	`5672`	AMQP	localhost:5672 — used by the Spring Boot app
RabbitMQ Management UI	`15672`	HTTP	http://localhost:15672 — queue/exchange/binding inspection
Mailhog SMTP	`1025`	SMTP	localhost:1025 — catches all outbound emails from the app
Mailhog Web UI	`8025`	HTTP	http://localhost:8025 — view sent emails in a browser
Redis Commander	`8081`	HTTP	http://localhost:8081 — inspect all Redis keys and values
pgAdmin (optional)	`5050`	HTTP	http://localhost:5050 — visual PostgreSQL query tool

11.2 Docker Compose — Complete Service Configuration

The complete `docker-compose.yml` defines 7 services. The `app` service depends on `postgres`, `redis`, and `rabbitmq` with condition `service_healthy` — this guarantees all three infrastructure services are fully ready before Spring Boot starts.

Service	Image	Ports	Volume	Key Config
app	`(local build)`	`8080:8080`	—	Reads env vars for all connection strings. Health check: curl -f http://localhost:8080/actuator/health every 30s.
postgres	`postgres:15-alpine`	`5432:5432`	`postgres_data:/var/lib/postgresql/data`	DB: ticketing, User: ticketing, Pass: ticketing_pass. Health check: pg_isready -U ticketing every 10s.
redis	`redis:7-alpine`	`6379:6379`	`redis_data:/data`	Command: redis-server --appendonly yes (enables AOF persistence so data survives container restart). Health check: redis-cli ping every 10s.
rabbitmq	`rabbitmq:3.12-management-alpine`	`5672:5672` `15672:15672`	`rabbitmq_data:/var/lib/rabbitmq`	Default user: ticketing / ticketing_pass. Management plugin built in. Health check: rabbitmq-diagnostics ping every 30s.
mailhog	`mailhog/mailhog`	`1025:1025` `8025:8025`	—	No configuration needed. Catches all SMTP traffic from JavaMailSender.
redis-commander	`rediscommander/redis-commander`	`8081:8081`	—	Env: REDIS_HOSTS=local:redis:6379.

App Service Environment Variables

The `app` service reads all secrets from the host environment (or a `.env` file in the project root, which must be in `.gitignore`). The following variables must be set before running `docker-compose up`:

Variable	Example Value	Description
`SPRING_DATASOURCE_URL`	`jdbc:postgresql://postgres:5432/ticketing`	Uses the Docker service name postgres as the hostname
`SPRING_DATASOURCE_USERNAME`	`ticketing`	Matches POSTGRES_USER in the postgres service
`SPRING_DATASOURCE_PASSWORD`	`ticketing_pass`	Matches POSTGRES_PASSWORD in the postgres service
`SPRING_DATA_REDIS_HOST`	`redis`	Uses the Docker service name redis as the hostname
`SPRING_RABBITMQ_HOST`	`rabbitmq`	Uses the Docker service name rabbitmq as the hostname
`STRIPE_SECRET_KEY`	`sk_test_...`	From your Stripe dashboard — test mode only for local dev
`STRIPE_WEBHOOK_SECRET`	`whsec_...`	From stripe listen CLI output — rotates each session

Variable	Example Value	Description
`FRONTEND_URL`	`http://localhost:3000`	Used in Stripe success/cancel URL construction

> ✔ **TIP:** Add a Makefile target: make dev that runs docker-compose up --build -d and then tails the app logs. This single command replaces the full startup sequence.

11.3 Dockerfile — Multi-Stage Build

The `Dockerfile` uses a two-stage build to produce a lean production image (JRE only, not JDK — approximately 180MB vs 400MB). Dependencies are cached as a separate layer so rebuilds after source changes do not re-download Maven artifacts.

Stage 1 — Builder (eclipse-temurin:17-jdk-alpine)

1. **Copy Maven wrapper and pom.xml first (without src/):** Copy `.mvn/`, `mvnw`, and `pom.xml`. Run `./mvnw dependency:go-offline -B`. Because `src/` has not been copied yet, Docker caches this layer. On future rebuilds, if `pom.xml` has not changed, the dependency download is skipped entirely.
2. **Copy source and build:** Copy `src/`. Run `./mvnw package -DskipTests -B`. The `-DskipTests` flag skips test execution in the Docker build (tests run separately in CI). The `-B` flag enables non-interactive batch mode.

Stage 2 — Runtime (eclipse-temurin:17-jre-alpine)

3. **Create non-root user:** Run `addgroup -S spring && adduser -S spring -G spring`. Switch to this user with `USER spring:spring`. Running as root inside a container is a security risk — the non-root user limits blast radius if the container is compromised.
4. **Copy the JAR:** Use `COPY --from=builder /app/target/*.jar app.jar`. The `--from=builder` flag pulls from the first stage. Only the JAR is copied — the JDK, Maven cache, and source code are left behind.
5. **Set JVM flags in ENTRYPOINT:** `java -XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 -jar app.jar`. `-XX:+UseContainerSupport` makes the JVM respect the container's cgroup memory limit (not the host total). `-XX:MaxRAMPercentage=75.0` allocates 75% of the container's memory to the JVM heap.

11.4 Railway Production Deployment — Step by Step

Railway provides PostgreSQL, Redis, and RabbitMQ as managed plugins — no self-managed infrastructure needed. The free tier is sufficient for Phase 1A.

Initial Setup (one-time)

6. Create a Railway account at railway.app and create a new project.
7. Click **"Deploy from GitHub repo"**, connect your GitHub account, and select the `ticketing-platform` repository.
8. Add PostgreSQL: click **"+ New"** → **Database** → **PostgreSQL**. Railway auto-creates and injects the `DATABASE_URL` environment variable.
9. Add Redis: click **"+ New"** → **Database** → **Redis**. Railway auto-creates and injects `REDIS_URL`.
10. Add RabbitMQ: click **"+ New"** → **Database** → **RabbitMQ** (or use CloudAMQP free tier if Railway RabbitMQ is unavailable). Note the connection URL.

Environment Variables to Set in Railway Dashboard

Variable	Source	Notes
`SPRING_DATASOURCE_URL`	Railway PostgreSQL plugin	Copy from the PostgreSQL service Variables tab
`SPRING_DATASOURCE_USERNAME`	Railway PostgreSQL plugin	Copy from the PostgreSQL service Variables tab
`SPRING_DATASOURCE_PASSWORD`	Railway PostgreSQL plugin	Copy from the PostgreSQL service Variables tab
`SPRING_DATA_REDIS_URL`	Railway Redis plugin	Format: redis://default:{password}@{host}:{port}
`SPRING_RABBITMQ_URL`	Railway RabbitMQ / CloudAMQP	Format: amqp://user:pass@host:port/vhost
`STRIPE_SECRET_KEY`	Your Stripe dashboard	Use test key (sk_test_...) for Phase 1A
`STRIPE_WEBHOOK_SECRET`	Stripe dashboard Webhooks section	After adding the Railway URL as a webhook endpoint
`FRONTEND_URL`	Your Vercel deployment URL	Used to construct Stripe redirect URLs
`SPRING_PROFILES_ACTIVE`	prod	Activates production Spring profile (no local dev overrides)
`SPRING_FLYWAY_BASELINE_ON_MIGRATE`	true	Required if the Railway DB already has a partial schema — prevents migration conflict

Post-Deployment Verification Checklist

- Health check: `curl https://{your-app}.up.railway.app/actuator/health` → should return `{"status":"UP"}`.
- Database migration: check Railway logs for `Successfully applied N migrations` — Flyway runs all 9 migrations on first deploy.

- Swagger UI: open `https://{your-app}.up.railway.app/swagger-ui.html` in browser — all endpoints should be listed.
- Update Stripe webhook endpoint in the Stripe dashboard to: `https://{your-app}.up.railway.app/api/webhooks/stripe`. Event types: `checkout.session.completed` and `checkout.session.expired`.
- Test end-to-end: reserve a ticket, proceed to checkout, use test card `4242 4242 4242 4242`, and verify the confirmation page loads with a QR code.

⚠ **WARNING:** Never commit application-local.yml to the repository. This file contains hardcoded local credentials and must be listed in .gitignore. All production values go through Railway environment variables — never in source code.

11.5 CI/CD Pipeline Overview

Two GitHub Actions workflows live in `.github/workflows/`. They run sequentially: the CI pipeline must pass before the deploy pipeline triggers.

Workflow File	Trigger	Jobs
`ci.yml`	Push to main or develop; any pull request to main	Job 1 (test): spins up postgres:15, redis:7, rabbitmq:3.12-management as service containers, runs ./mvnw test -Dspring.profiles.active=test, uploads JaCoCo report as artifact. Job 2 (build): needs test, runs docker build to verify the image builds cleanly.
`deploy.yml`	Push to main only (after ci.yml passes)	Single deploy job: uses bervProject/railway-deploy@main action with RAILWAY_TOKEN from GitHub Secrets. Triggers Railway to pull the latest main and redeploy.

- **Maven cache:** Add `actions/cache@v3` with path `~/.m2` and key `${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}`. This caches the Maven local repository between workflow runs — saves 2–4 minutes per build.
- **RAILWAY_TOKEN:** Generate from Railway dashboard → Account → Tokens. Add to GitHub: repository Settings → Secrets and variables → Actions → New repository secret named `RAILWAY_TOKEN`.

SECTION 12

Fundamentals Track Detail

System design study approach — Design Ticketmaster and Design Eventbrite — with full capacity estimation and architecture guidance

The fundamentals track runs in parallel with building the project. The goal is to develop **genuine system design intuition** — not to memorise answers, but to understand **why** each architectural decision was made. Because you are building a ticketing platform, the Design Ticketmaster and Design Eventbrite exercises are not hypothetical — you have **direct experience** with the exact trade-offs being discussed.

12.1 How to Approach Each System Design Exercise

Each exercise is a **45-minute timed session**. Stop at 45 minutes even if incomplete — the constraint forces prioritisation, which is the skill being developed.

Minutes	Phase	What to Produce
0–5	**Clarify scope**	Write 3–5 clarifying questions and answer them yourself. Example: Is this read-heavy or write-heavy? Are we designing the ticket browsing, the purchase flow, or both?
5–15	**Capacity estimation**	Calculate users, QPS, storage, and bandwidth. Write numbers down — even rough estimates reveal where scale problems are.
15–30	**Architecture diagram**	Draw the high-level diagram on paper or Excalidraw: CDN, API Gateway, Load Balancer, App Servers, Databases, Cache, Queue, Storage. Label each component.
30–40	**Deep dive on hardest part**	Zoom in on the most technically interesting component. For ticketing: the seat reservation and lock mechanism. Write the data model and the key algorithm.
40–45	**Trade-offs and bottlenecks**	Write 3 trade-offs you made and identify the single biggest bottleneck. What would fail first at 10× the load?

12.2 Design Ticketmaster (Week 2, Day 14 — 45 min timed)

Design a high-concurrency event ticketing platform handling peak loads (e.g. Taylor Swift tickets). This is the **system you are building in Phase 1A** — you have direct insight into the trade-offs.

Architecture Components to Draw

- **CDN:** Serves static assets (event images, CSS, JS bundles). Reduces load on app servers for 90% of requests. Use CloudFront or Cloudflare.
- **API Gateway:** Rate limiting at the edge — critical for ticket rush. Limits each user to e.g. 10 requests/second to prevent bot flooding. Also handles auth token validation, request routing, and HTTPS termination.
- **Load Balancer → Auto-scaling App Server Pool:** Routes requests across stateless app server instances. Stateless is critical — any server can handle any request; session state lives in Redis, not on the server.
- **Booking Service (stateless):** The core reservation API. Horizontally scalable. Acquires Redis lock before touching inventory. The only state it holds is in the request context — nothing persisted in-process.
- **PostgreSQL Primary + 2 Read Replicas:** Bookings and payments always go to the primary (write path). Event browsing, category listing, and search queries go to read replicas (read path). Replicas lag ~10ms behind primary — acceptable for read use cases.
- **Redis Cluster (3 nodes):** Distributed locks (seat reservation), cart data, tier availability counts, event list cache. Three nodes provide availability — if one node fails, the cluster remains operational.
- **RabbitMQ:** Decouples confirmation emails and PDF generation from the payment response. A slow email provider cannot delay the checkout API response.
- **S3 / Object Storage:** Stores ticket PDFs generated after booking confirmation, event cover images uploaded by organizers. Offloads binary storage from the database.

Capacity Estimation — Taylor Swift Peak Event

Metric	Estimate	Reasoning
Registered users	**10 million**	Global platform baseline
Daily active users (DAU)	**100,000**	Normal day — browsing events
Peak concurrent users (ticket rush)	**50,000**	Taylor Swift on-sale — everyone hits Buy at 10:00 AM simultaneously
Events per day	**1,000**	Mix of large and small events
Avg tickets per event	**5,000**	Across all venue sizes
Peak booking API QPS	**50,000 req/sec**	50K concurrent users × 1 req each at the moment tickets go on sale
App servers needed at peak	**50 servers**	Each server handles ~1,000 req/sec (Spring Boot under load)

Metric	Estimate	Reasoning
DB read QPS (event browsing)	**200,000/sec**	Browsing is 4× more frequent than purchasing — served by read replicas
DB write QPS (ticket rush)	**50,000/sec**	Every booking is a write — goes to primary only. Consider Vitess/sharding at this scale.
Redis lock operations/sec	**50,000/sec**	One lock acquisition per reservation attempt — Redis Cluster handles this comfortably
Storage growth (bookings)	**5 GB/day**	5M tickets × ~1KB per booking row
Email volume (RabbitMQ)	**5M messages/day**	One confirmation email per ticket sold

Key Architecture Questions — Write 1–2 Sentences Each

Question	Recommended Answer
How do you prevent overselling when 100,000 users hit "Buy" simultaneously?	Acquire a Redis distributed lock (SET NX EX) per tier before decrementing availability. The lock ensures only one thread modifies the count at a time. Optimistic locking (@Version on Booking entity) provides a second safety net.
Why Redis locks instead of database-level locks?	Database row-level locks hold a DB connection for the lock duration — at 50K QPS this would exhaust the connection pool. Redis locks are in-memory, sub-millisecond, and do not hold DB connections. They scale independently.
Why is the booking service stateless? What state does it NOT store?	It does not store session data, user authentication state, or cart data in-process. All of this lives in Redis (cart hash, JWT validated per-request). Statelessness allows horizontal scaling — any server handles any request.
What happens if the Redis node holding the lock crashes during a purchase?	With a single-node Redis, the lock is lost and the seat could be double-sold. Mitigation: use a Redis Cluster (3+ nodes, Redlock algorithm). For Phase 1A, single-node is acceptable — document this as a known production trade-off.
How do you handle email confirmation if RabbitMQ goes down?	RabbitMQ is on the async path — it does not affect booking confirmation. The booking is CONFIRMED before the message is published. If RabbitMQ is down, the message is buffered locally and retried. If the queue is lost, the user does not receive an email but their booking is confirmed — acceptable for MVP.
What DB index would you add for "find all upcoming concerts in New York"?	A composite index on events(status, start_date, venue_id) — or a partial index WHERE status='PUBLISHED'. Join to venues and filter on city. EXPLAIN ANALYZE should show Index Scan, not Seq Scan.

Bottleneck Analysis

- **Primary bottleneck:** The Redis lock acquisition is the critical path. Every reservation must acquire a lock — this serialises access to each tier. At 50K concurrent users competing for the same tier, 49,999 of them are blocked waiting. This is intentional — it prevents overselling — but it means lock throughput is the ceiling.

- **Secondary bottleneck:** The PostgreSQL primary write path. 50K writes/second is near the limit of a single PostgreSQL instance. Solutions: connection pooling via PgBouncer, write batching, or eventually Vitess for horizontal sharding.
- **What to mention in an interview:** The seat reservation problem is fundamentally a distributed coordination problem. Every solution involves a trade-off between consistency (no overselling), availability (always accept requests), and latency (fast response). Redis gives us consistency + low latency at the cost of availability if the cluster is down.

12.3 Design Eventbrite (Week 3, Day 19 — 45 min timed)

Design a self-service event creation and ticketing platform for organizers. Eventbrite is **organizer-centric** (anyone can create an event) vs Ticketmaster which is **venue-centric** (events are created by large venues with dedicated sales teams). The architectural differences flow directly from this distinction.

How Eventbrite Differs from Ticketmaster

Dimension	Ticketmaster	Eventbrite
Primary user	Consumer (buyer)	Organizer (seller) + Consumer (buyer)
Event creation	Enterprise venue operators only	Any registered user (organizer role)
Free events	Rare — all events are paid	Very common — most small events are free
Payment flow	Always present (Stripe)	Optional — free events skip payment entirely
Scale requirement	Massive peak (50K concurrent for one event)	Many small events simultaneously (long tail)
Check-in mechanism	Ticket scan at a large venue gate	Organizer scans QR codes at the door via mobile app
Refund policy	Complex, event-specific	Organizer-defined per event
Core API focus	Low-latency reservation under peak concurrency	Self-service CRUD for events + embeddable ticket widget

Architecture Components to Draw

- **Organizer Portal (SPA):** React/Next.js dashboard for event creation, ticket tier configuration, attendee management, revenue analytics. This is a distinct user flow from the consumer-facing pages.
- **Event Creation Service:** Handles the CRUD for events. Organizers can draft, publish, and close events. Includes image upload to S3, slug generation, and category assignment.

- **Embeddable Ticket Widget:** A JavaScript snippet that organizers paste into their own website. Renders a ticket purchase flow inside an iframe. Requires a separate lightweight API that returns session-based Stripe Checkout URLs without the full app context.
- **Free Event Flow (no payment path):** When an organizer marks all tiers as free, the checkout flow skips Stripe entirely. The booking transitions directly from RESERVED → CONFIRMED on "RSVP" button click. Email confirmation is still sent via RabbitMQ.
- **QR Code Check-in API:** A mobile-optimized endpoint `POST /api/events/{id}/check-in` that accepts a QR code payload, verifies the signature, looks up the ticket, validates it is CONFIRMED and not already checked in, and sets `check_in_status=true` and `check_in_time=now`.
- **Analytics Service:** Revenue totals per event, ticket sales over time, attendee demographics. Read-only against a read replica. Pre-aggregated by a nightly batch job for faster dashboard loads.

Capacity Estimation — Eventbrite Scale

Metric	Estimate	Reasoning
Registered organizers	**500,000**	Global — anyone can create an event
Active events (live, on-sale)	**200,000**	Long tail of small events running simultaneously
Avg peak concurrent users per event	**50–500**	Most events are small — not 50K like Taylor Swift
Total platform peak QPS	**100,000 req/sec**	Distributed across 200K events — no single event dominates
Free event RSVPs	**70% of all bookings**	No payment processing needed — simpler, faster flow
Paid event bookings	**30% of all bookings**	Full Stripe checkout flow
Check-in API calls	**10 million/day**	Organizers scan attendees at door — bursty at event start time
Event image storage	**1 TB/year**	Organizer-uploaded event banners and gallery images
Analytics queries	**500,000/day**	Organizer dashboard page loads — served from read replica

Key Architecture Questions — Write 1–2 Sentences Each

Question	Recommended Answer
How do you handle free events differently from paid events in the booking flow?	Free events skip the payment state machine path entirely. On "RSVP" the booking transitions RESERVED → CONFIRMED directly without creating a Stripe session. The QR code is generated immediately and the confirmation email is sent.
How does the check-in API prevent the same QR code being scanned twice?	The check-in API is idempotent for the same ticketId but uses a database transaction with a SELECT FOR UPDATE on the ticket row.

Question	Recommended Answer
	If check_in_status is already true, return 409. The database constraint ensures no concurrent double-scan.
How would you design the embeddable ticket widget for organizer websites?	Serve a lightweight <script> that calls a public endpoint /api/widgets/{eventId} to get available tiers. On "Buy" click, make a POST to create a session and redirect to Stripe Checkout. The widget iframe is sandboxed — no access to the parent page DOM.
How do you scale the check-in API for an event with 50,000 attendees all arriving in 30 minutes?	The check-in API is write-heavy and bursty. Use a write-through cache: update Redis first (SET ticket:{id}:checked_in 1 NX) for instant conflict detection, then write to the DB asynchronously. Redis SET NX is the atomic gate preventing double check-in.
What is the key system design difference between Ticketmaster and Eventbrite that affects the database schema?	Ticketmaster: events are tied to venues (venue-centric). Eventbrite: events are created by organizers and can be anywhere (organizer-centric). Eventbrite needs a richer organizer profile, event-level refund policy configuration, and the concept of a "free" ticket type with zero price.
How would you build the analytics dashboard without slowing down the OLTP database?	Run analytics queries on a read replica, not the primary. For complex aggregations (revenue over time, funnel analysis), run a nightly batch job (Spring Batch) that pre-computes summaries into an analytics table. The dashboard reads the pre-computed table — millisecond response time.

Trade-offs vs Ticketmaster Design

- **Trade-off 1 — Consistency vs Availability for free RSVPs:** Free events do not require a distributed lock — the RSVP is cheaper. But if 2,000 people RSVP for a 100-person event, you still need capacity enforcement. Use optimistic locking (`@Version` on the TicketTier `available_count`) rather than Redis locks for free events — simpler, and the throughput requirement is lower.
- **Trade-off 2 — Organizer flexibility vs system complexity:** Allowing organizers to configure their own refund policies (per-event refund windows, percentage rules) requires a `RefundPolicy` entity linked to each event, instead of the global 3-tier rule in Phase 1A. More flexible, more complex — Phase 1B candidate.
- **Trade-off 3 — Embeddable widget security vs feature richness:** The embeddable widget must be sandboxed (iframe with Content-Security-Policy) to prevent organizer websites from intercepting payment data. This limits the widget to a redirect-to-Stripe flow — you cannot embed Stripe Elements directly in the iframe without a more complex integration.

12.4 System Design Study Resources

Use these resources to deepen understanding between design sessions:

Resource	Format	What to Focus On
Designing Data-Intensive Applications (DDIA) — Ch. 1–3	Book	Reliable, Scalable, Maintainable Systems; Data Models; Storage and Retrieval. The foundational mental models.
systemdesign.one/ticketmaster-system-design	Article	Reference implementation for the Design Ticketmaster exercise. Compare with your own diagram.
ByteByteGo — "Redis Distributed Lock Explained"	YouTube (~12 min)	Visual walkthrough of SETNX vs SET NX EX, the Lua release script, and Redlock. Directly relevant to Phase 1A.
ByteByteGo — "Design a Ticketing System"	YouTube (~15 min)	High-level walkthrough — use after completing your own design to see alternative approaches.
High Scalability Blog — real-world architecture teardowns	Blog	Read 2–3 articles on systems you have used. Understanding real production systems builds intuition faster than hypothetical examples.
Grokking System Design (Educative, if you have access)	Online course	Structured problems with solutions. Use for additional exercises beyond Ticketmaster and Eventbrite.

▸ **RULE:** After each system design session: write down the single most important thing you learned or the biggest gap you identified. Review these notes before interviews — they are your personalised interview talking points.

Phase 1A • Sections 10, 11 & 12 • April 4–24, 2026 | See Section 2 for the day-by-day execution map | See Sections 6–9 for infrastructure implementation guides